The Journal of Supercomputing
https://doi.org/10.1007/s11227-021-03807-3

An energy-efficient cuckoo search algorithm for virtual machine placement in cloud computing data centers

Hamza Onoruoiza Salami[1] · Abubakar Bala[2,3] · Sadiq M. Sait[4] · Idris Ismail[2]

Accepted: 9 April 2021
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2021

Abstract
The advent of virtualization technology has created a huge potential application for cloud computing. In virtualization, a large hardware resource is often broken down into smaller virtual units. These small units are then provisioned to different clients. However, these services need to be provided in such a way that resources are properly utilized. To achieve this, many of the scheduling, allocation, and provisioning issues of data centers are formulated as optimization problems. The virtual machine placement problem (VMPP) is a typical provisioning problem of data centers. In VMPP, several virtual machine requests are to be hosted on physical machines such that a minimum number of physical machines are used. This work proposes a cuckoo search (CS) inspired algorithm for solving the VMPP. To improve the algorithm's performance, new cost and perturbation functions are developed. The proposed method was tested on two well-known benchmark datasets. It outperformed the reordered grouping genetic algorithm, best-fit decreasing, first-fit decreasing, and an earlier CS method called multiCSA.

Keywords Bin-packing · Cloud computing · Combinatorial optimization · Cuckoo search · Data center energy management · Lévy flight · NP-Hard

1 Introduction

Cloud computing is a relatively novel model that has gotten considerable attention from researchers and industry practitioners [1]. It could be described as a computing model that grants remote services to different clients via an Internet connection. These services are often located in data centers. Like public utility service, cloud computing can also be purchased on a pay-as-you-use basis [2]. The provisioning paradigm of cloud computing can be broadly divided into three classes

✉ Hamza Onoruoiza Salami
hamzaosalami@gmail.com

Extended author information available on the last page of the article

Published online: 23 April 2021

[3]: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). In IaaS, hardware elements such as processors, storage, and network devices are provided as a service to clients. In contrast, PaaS contains the likes of development tools, database management systems, and middleware. Lastly, SaaS provides clients with access to software on the cloud without the need to install or purchase the real license of the software product. Recently, most cloud providers give a combination of these services. These services rely on virtualization technology [3]. In this technology, resources are divided into many smaller subvirtual units, e.g., a physical computer may be divided into several virtual machines (VMs), with each VM operating almost autonomously. Each of these VMs may then be provided as a service to clients.

Most cloud computing problems may be formulated as traditional optimization problems, especially resource provisioning and scheduling. The main aim of the optimization is to improve resource utilization for reduced costs and optimal service delivery. This paper focuses on the resource provisioning problem of cloud computing. Here, there are several virtual machine requests, and the optimization aims to place all these VMs on physical machines (PMs) so that a minimum number of PMs are used. This translates into optimal resource utilization and is a variant of the famous bin packing problem (BPP) [4]. In BPP, small items must be placed in bins such that the minimum number of bins is used. This problem has been proven to be an NP-Hard problem [5]. Here our resources are twofold: CPU and memory. Thus, smaller VMs with CPU and memory requirements are placed in larger physical machines, such that a minimum number of physical machines are used. The problem is termed: virtual machine placement problem (VMPP). It is an optimization problem, and thus many metaheuristics can be used to solve it. One of these metaheuristics is the cuckoo search optimization algorithm. The cuckoo search algorithm (CSA) is a relatively new metaheuristic optimization algorithm introduced by Yang and Deb [6]. CSA is inspired by the brood parasitism of some cuckoos species, such as the *ani* and *Guira* cuckoos. These cuckoos exhibit an aggressive reproduction strategy by laying their eggs in the nests of other host birds. Upon discovering the eggs that are not theirs, host birds may throw away those eggs or abandon their nests to build new nests in different locations. The standard CSA is based on three idealized rules [6]: (i) each cuckoo lays one egg at a time and drops it in a randomly selected nest. (ii) the best nests with high-quality solutions move over to the next generation. (iii) there are a fixed number of host nests, and the egg laid by a cuckoo is discovered by a host with probability $p \epsilon [0, 1]$. Once a host discovers a foreign egg, it throws away the egg or abandons its nest to build a completely new nest. This last step can be implemented by replacing a proportion p of the n nests with new solutions. Many variants of the CSA exist, and among them, the modified cuckoo search (MCS) algorithm of Walton et al. [7] stands out. Thus, the improved CSA for solving VMPP proposed here is based on the MCS of Walton et al. [7].

The main contributions of this research are:

- The development of a new cost function for the placement solution.
- Proposal of three new perturbation functions for searching the design space.

– A new technique of updating the cost of solutions that is computationally cheap.

The rest of the paper is outlined as follows: Sect. 2 presents a brief literature review. Section 3 discusses the virtual machine placement problem. In Sect. 4, the methods of the new cuckoo search algorithm for virtual machine placement are outlined. Section 5 presents and analyses the results of simulation experiments. Finally, Sect. 6 gives concluding remarks.

2 Related work

Since the VM placement problem (VMPP) is a type of the ancient bin packing problem (BPP), many exact methods and metaheuristics developed to solve the BPP can solve the VMPP. Our work focuses on two dimensions (memory and CPU). Thus it falls in the category of two-dimensional bin packing problem (2DBPP). Many techniques have been proposed to solve the 2DBPP. These works include those of Cid-Garcia et al. [8], where a two-step solution was proposed. The first step generates possible valid positions for items in the bin, while the second stage selects the optimal item positioning. Another new method for solving the 2DBPP is that of Ross et al. [9], where a human-derived heuristic (HDH) was combined with a hybrid evolutionary algorithm. The HDH used a decision tree to replace the mutation operator of the evolutionary algorithm. Researchers in [10] developed a branch-and-price method to solve the 2DBPP. Other techniques for solving the 2DBPP include [11–13].

However, other methods have been used across different dimensional bin packing problems, such as the first-fit decreasing (FFD) algorithm. In the FFD, items (in our case, VMs) are sorted in decreasing order of sizes (in our case, resource utilization requests), and each of the sorted VMs is placed in the next available bin (in our case, physical machine (PM)). If a VM cannot be fit in any of the existing PMs, a new PM is created, and the VM is placed in it. The process continues till all the VMs are placed. Lately, a new variant of the FFD was proposed by Mustafa et al. [14] for solving online VM placement problems.

Since our technique revolves around using metaheuristics to solve the virtual machine placement problem in data centers, the reminder of this section focuses on works that do similar tasks. For instance, in [15], the VMPP was formulated as a multiobjective optimization problem, and the famous NSGA-II was employed to tackle it. Also, Xu et al. [16] developed a VM placement technique by hybridizing genetic algorithm (GA) with fuzzy logic. Moreover, in [17], a hybrid of firefly algorithm (FA) and fuzzy was used for VM placement. Similarly, Wu et al. [18] applied simulated annealing (SA) to minimize power consumption in data centers. In [19], an ant colony optimization (ACO) algorithm was proposed for the placement of VMs, targeting minimization of resource wastage and reduction in power consumption. In Wang et al. [20] a particle swarm optimization (PSO) technique was proposed for VM placement to minimize energy. Other works that use metaheuristics to solve the VMPP include biogeography-based optimization by Alboaneen et al. [21], and Glowworm swarm optimization (GSO) in [22]. Furthermore, Abdel-Baset et al.

[23] developed a Lévy-based whale optimization algorithm for the VMPP. Similarly, Li et al. [24] developed a chemical reaction optimization method for solving the VMPP. They minimized the energy consumption while at the same time improving resource utilization.

Sait et al. [25] developed a multiobjective cuckoo search (CS) optimization algorithm to solve the VMPP. They simultaneously minimized power consumption and resource wastage. This work also proposes an improved cuckoo search algorithm for VM placement. The enhancement over the work in [25] is three-fold. First, a new cost function is developed. Moreover, three new perturbation functions are proposed. Finally, we propose a new technique of updating the cost of perturbed solutions from the parent solutions. These three improvements helped the new CS algorithm to obtain better results in a very short time.

3 Virtual machine placement problem

The Virtual Machine Placement Problem (VMPP) involves placing each virtual machine on a physical server such that the minimum number of servers are utilized, and no server is overloaded (i.e., the sum of resource needs of all virtual machines placed on each server does not exceed the maximum resources available on the server). Let $v_1, v_2, ...v_N$ be N virtual machines to be placed on a set of M servers $s_1, s_2, ...s_M$. Each virtual machine v_j requires cv_j units of CPU and mv_j units of memory. Each server s_i has a CPU capacity of CS_i and memory capacity MS_i. We define two binary variables X_{ij} and y_i $(i = 1, 2, ...M; j = 1, 2, ...N)$. If the virtual machine v_j is placed on server $s_i, X_{ij} = 1$. Otherwise, $X_{ij} = 0$. If s_i is active (i.e., if s_i hosts at least one virtual machine), $y_i = 1$. On the other hand, if s_i is idle, $y_i = 0$. Figure 1 shows how two VMs are packed into a single server. The vertical dimension of the server represents CPU utilization, while the horizontal dimension represents memory utilization. Furthermore, Fig. 2 shows three different ways of placing five VMs $(v_1, v_2...v_5)$ on three servers (s_1, s_2 and s_3). The CPU and memory requirements of $v_1, ...v_5$ are (50, 40), (30, 10), (10, 40), (60, 40) and (30, 50), respectively, while the CPU and memory capacities for each server are (90, 90). Thus, the CPU and memory utilization of s_1, s_2 and s_3, respectively, is (70,

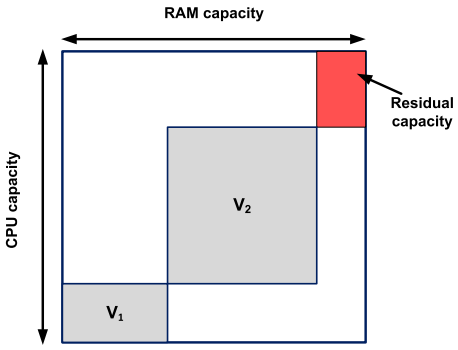
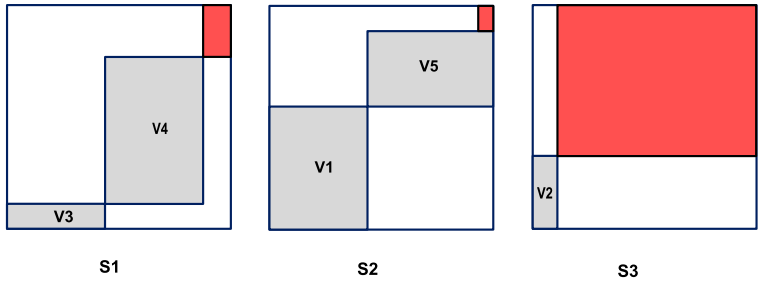
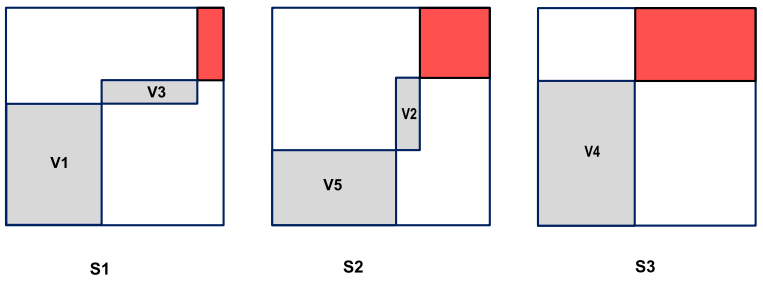


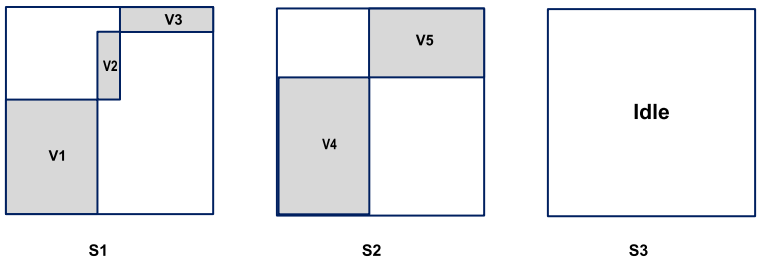
Fig. 1 An example of VM placement on a single server [25]



(a) Placement 1



(b) Placement 2



(c) Placement 3

Fig. 2 Three possible ways of placing five virtual machines on three servers, with placement 3 given the best arrangement

80), (80, 90), (30, 10) for Fig. 2a; (60, 80), (60, 60), (60, 40) for Fig. 2b; and (90, 90), (90, 90), (0, 0) for 2c, respectively. Note that the placement of Fig. 2c represents a perfect placement of the five VMs, since there are no wasted resources on the first two servers, while the third server is idle and thus can be switched off.

The virtual machine placement problem can be formulated as:

$$\text{Minimize} f(y) = \sum_{i=1}^{M} y_i \tag{1}$$

Subject to:

$$\sum_{i=1}^{M} X_{ij} = 1 \qquad j = 1, 2, ...N \tag{2}$$

$$\sum_{j=1}^{N} X_{ij} \cdot \mathrm{cv}_j \leq \mathrm{CS}_i \qquad i = 1, 2, \ldots M \tag{3}$$

$$\sum_{j=1}^{N} X_{ij} \cdot \mathrm{mv}_j \leq \mathrm{MS}_i \qquad i = 1, 2, \ldots M \tag{4}$$

$$X_{ij}, y_i \ \epsilon \ \{0, 1\} \qquad i = 1, 2, ...M, \qquad j = 1, 2, ...N \tag{5}$$

The constraint in (2) ensures that each virtual machine is placed on exactly one server. Constraints (3) and (4) guarantee that the CPU and memory capacity of each server is not exceeded. The constraints in (5) define X_{ij} and y_i as binary variables. Since an idle server that is turned on consumes 50%–70% of the energy consumed by a fully loaded server, the total power consumed by servers can be cut down by efficiently placing VMs on servers and shutting down servers that do not host any VM [26]. The power consumed by an active server s_i can be computed using (6) [25]:

$$P(s_i) = P_{\mathrm{idle}} + (P_{\mathrm{max}} - P_{\mathrm{idle}}) \cdot \mathrm{CU}_i / \mathrm{CS}_i \tag{6}$$

where CU_i represents the current CPU utilization of s_i while P_{max} and P_{idle} denote the average power consumption of a server when it is fully utilized and when it is idle but active, respectively. Note that in this paper, an idle server does not consume any power because it is assumed to be shut down.

4 New cuckoo search algorithm for virtual machine placement

This section details the proposed CSA for solving the VMPP. The major components of the CSA such as solution representation, cost/fitness computation, and perturbation functions are first described before the entire algorithm is discussed.

4.1 Solution representation

Each nest (solution) is an N-dimensional vector in which the jth value indicates which server hosts virtual machine v_j. Thus, the placements in Fig. 2a–c can be represented compactly as (2, 3, 1, 1, 2), (1, 2, 1, 3, 2) and (1, 1, 1, 2, 2), respectively.

4.2 Cost functions

The cost of the placements on a server s_i can be computed using (7), where CU_i and MU_i represent the current CPU and memory utilization of s_i, respectively. The first case of (7) shows that the cost of placement on an idle server is zero. In the second case, an overloaded server's cost is the same as the number of VMs, i.e., N. The choice of this value will be explained shortly. In the third case, the server is well-placed (i.e., it is not overloaded). Thus, the placement cost is computed as the square root of the average of the proportion of leftover CPU and memory resources for the server. Note that a perfectly filled server (i.e., where $\mathrm{CU}_i = \mathrm{CS}_i$ and $\mathrm{MU}_i = \mathrm{MS}_i$) has no leftover resources; thus, it has an optimum cost of zero.

$$c(s_i) = \begin{cases} 0 & \mathrm{CU}_i = 0 \text{ and } \mathrm{MU}_i = 0 \\ N & \mathrm{CU}_i > \mathrm{CS}_i \text{ or } \mathrm{MU}_i > \mathrm{MS}_i \\ \sqrt{\frac{1}{2}\left(\frac{\mathrm{CS}_i - \mathrm{CU}_i}{\mathrm{CS}_i} + \frac{\mathrm{MS}_i - \mathrm{MU}_i}{\mathrm{MS}_i}\right)} & \text{otherwise} \end{cases} \tag{7}$$

Furthermore, the cost of a solution is the sum of the cost of placements of all servers as shown in (8):

$$c(\mathrm{nest}) = \sum_{i=1}^{M} c(s_i) \tag{8}$$

Using (7) and (8), the costs of the solutions in Fig. 2 are computed as follows:

Fig. 2a: $c(\mathrm{nest1}) = \sqrt{15/90} + \sqrt{5/90} + \sqrt{70/90} = 1.53$.
Fig. 2b: $c(\mathrm{nest2}) = \sqrt{20/90} + \sqrt{30/90} + \sqrt{40/90} = 1.72$.
Fig. 2c: $c(\mathrm{nest3}) = \sqrt{0} + \sqrt{0} + \sqrt{0} = 0$.

The following points are worth noting about the cost functions of (7) and (8):

- Because the cost of a solution is simply the sum of the cost for all servers, when a solution changes, (8) is not required to recompute the cost of the new solution afresh. Instead, the cost of the solution is obtained by computing the change in cost for only the affected servers. For example, if a solution {1, 2, **2**, 2, 1} changes to {1, 2, **3**, 2, 1}, only servers s_2 and s_3 are affected. Using, (7), the change in the cost of these two servers can be computed and used to obtain the new solution's cost. This technique of bypassing the use of (8) leads to tremendous speedups when there are tens or hundreds of servers.
- Since the cost for a single well-placed server lies between [0, 1), the cost of a well-placed solution (i.e., one without overloaded servers) lies in the range [0, N) even in the worst-case scenario where every VM is placed on a separate server. On the other hand, an overloaded server has a cost of N. Thus, a solution in which there is at least one overloaded server will always have a higher cost than one in which each server is well-placed.

– The square-root operation in (7) helps to differentiate between two nests having equal numbers of well-placed servers and idle servers. It ensures that nests in which few servers have large leftover resources yield lower costs than nests in which small leftover resources are scattered among many servers. For example, nest 1 of Fig. 2a has a lower cost than nest 2 of Fig. 2b because s_3 of the former has large enough resources that can likely accommodate other VM(s) in the future, whereas, in the latter, the remaining resources are scattered into smaller bits among the three servers. It is noteworthy that without the square-root operation of (7), nest 1 and nest 2 would have a cost of 1.0.

4.3 Population initialization

The (*Init*) function is used to initialize each nest in the population. The function begins by generating a random permutation of N numbers, representing the N VMs. Each VM in the permutation is considered in turn, and adjacent VMs in the permutation are placed in the same server if possible. Otherwise, the current VM is placed in the next idle server. Thus, the first VM in the permutation is placed in server s_1, the second VM in the permutation is placed in s_1 if possible. Otherwise, it is placed in s_2. The third VM in the permutation is placed in the same server as the second VM if possible or placed in the next empty server, and so on. For example, consider the VMs of Fig. 2. Assuming the random permutation generated is $\{1, 2, 4, 5, 3\}$, the VMs are placed in servers as follows: v_1 and v_2 are placed in s_1, v_4 cannot be accommodated in s_1, so it is placed in s_2 and v_5 joins v_4 in s_2. Finally, v_3 is placed in s_3 since it cannot be accommodated in s_2. Consequently, the initial nest becomes $\{1, 1, 3, 2, 2\}$.

4.4 Perturbation functions

Three different perturbation functions are used in the proposed CSA. Each function uses a modified version of the first fit decreasing (FFD) heuristic [27]. The three functions are quite similar, but they vary in two aspects: (i) whether VMs or servers are selected; and (ii) how VMs are sorted.

4.4.1 Modify$_1$

A set of a VMs in a solution is selected at random and removed from their current servers. The affected servers are then sorted in decreasing order of total resource utilization (i.e., in decreasing order of $\mathrm{CU}_i + \mathrm{MU}_i$). Next, the selected VMs are also sorted in decreasing order of total resource needs (i.e., in decreasing order of $\mathrm{cv}_j + \mathrm{mv}_j$). Each VM (in sorted order) is placed in the first server that can accommodate it. If none of the affected servers can accommodate a VM, it is placed in a server that is randomly chosen from the set of all servers, possibly resulting in over-utilization of the chosen server. However, the cost of any overloaded server will be penalized in (7). Note that if any of the initially selected a VMs is found to belong to a perfectly filled server, the VM is discarded, and another one is randomly selected.

4.4.2 Modify$_2$

A set of b servers is selected at random, and their currently assigned VMs are removed. Thus, each of the b servers now has $\mathrm{CU}_i = 0$ and $\mathrm{MU}_i = 0$. All the affected VMs are sorted in decreasing order of total resource needs (i.e., in decreasing order of $\mathrm{cv}_j + \mathrm{mv}_j$). Each VM (in sorted order) is placed in the first server that can accommodate it. If no server can accommodate a VM, it is placed in a server that is randomly chosen from the set of all servers, possibly resulting in overutilization of the chosen server's resources. Note that if any of the initially selected b servers is perfectly filled, the server is discarded, and another one is randomly selected. Furthermore, the selected servers are not ordered initially because they have been emptied, and in the case of homogeneous servers, their capacities are identical.

4.4.3 Modify$_3$

This function is very similar to Modify$_2$, except that the affected VMs are sorted in decreasing order of the difference between CPU and memory requirements for the VM (i.e., in decreasing order of $|cv_j - mv_j|$. Thus, while Modify$_2$ begins by considering VMs having large resource needs, Modify$_3$ begins with VMs whose resource needs are disproportionate (large CPU and small memory need, or small CPU and large memory need) and are thus likely to cause wastage of server resources.

In each of the perturbation functions described above, VMs are transferred between a few servers. To minimize computational overhead, we obtain the cost of the modified solution by applying (7) to determine the change in cost of only the affected servers. Therefore, there are two outputs for each perturbation function: the new solution and the total change in the solution's cost. Table 1 elaborates the differences between the perturbation functions and FFD heuristic.

4.5 Termination conditions

The CSA terminates if: (i) a maximum number of generations (*MAXGen*) is reached; (ii) a nest with the optimal cost of zero is obtained, indicating that there are no underutilized resources on any server because all are servers are perfectly filled; or (iii) the best cost does not improve within a set number of generations (*StallGens*), indicating that the CSA might have finally converged.

4.6 Complete algorithm

The proposed CSA is shown in Fig. 3. Lines 3–6 set the values of important parameters used in the algorithm. Line 7 generates the initial nest population using the aforementioned *Init* function. Subsequently, line 8 uses (8) to compute the cost of each nest in the population. In each generation, nests are partitioned into top and bottom nests based on their costs (Lines 10 and 11). A fixed proportion (*fracBottom*) of nests having the highest costs constitutes the bottom nests, while the remaining

Table 1 Comparison of FFD heuristic and the perturbation functions

	FFD	Modify_1	Modify_2	Modify_3
Initial	All servers are initially empty	All servers are initially nonempty, then a few randomly chosen VMs are removed from their current servers	All servers are initially nonempty, then a few randomly chosen servers are emptied	All servers are initially nonempty, then a few randomly chosen servers are emptied
Sorting of VMs	VMs are sorted in decreasing order of CPU needs (cv_j)	VMs are sorted in decreasing order of total resource needs ($\text{cv}_j + \text{mv}_j$)	VMs are sorted in decreasing order of total resource needs ($\text{cv}_j + \text{mv}_j$)	VMs are sorted in decreasing order of disproportionate resource needs ($\lvert cv_j - mv_j \rvert$)
Sorting of servers	No server is sorted	Selected servers are sorted in decreasing order of total resource utilization ($\text{CU}_i + \text{MU}_i$)	Selected servers are not sorted	Selected servers are not sorted

```
1: Inputs:
   Group of VMs [cv_j, mv_j], j ∈ 1, ... M
   a ← # of VMs to select in Modify_1
   b ← # of servers to select in Modify_2 and Modify_3
2: Outputs: Best nest, its cost, # of severs required & power consumption.

3: maxGen ← 2000 {Maximum # of generations}
4: stallGens ← 100 {Stop the CSA if no improvemnt in stallGens}
5: populationSize ← 50
6: fracBottom ← 0.3 {Fraction of population in bottom nests.}
7: Initialize the population using Init()
8: Find the cost of each nest

9: for G = 1 to maxGen do
10:    Sort nests in order of cost
11:    Group the nests into top and bottom nests

12:    for all X_i, member of bottom nests do
13:       [X_i, changeInCost] = Modify_1(X_i, a)
14:       C(X_i) = C(X_i) + changeInCost
15:    end for

16:    for all X_i, member of top nests do
17:       if rand(1.0) > 0.5 then
18:          [X_j, changeInCost] = Modify_2(X_i, b)
19:       else
20:          [X_j, changeInCost] = Modify_3(X_i, b)
21:       end if
22:       C(X_j) = C(X_i) + changeInCost
23:       Randomly select a nest (X_k) from the entire population
24:       if C(X_k) > C(X_j) then
25:          X_k ← X_j
26:          C(X_k) ← C(X_j)
27:       end if
28:    end for

29:    genBest(G) = best cost in iteration
30:    Update best nest found (bestNest) so far and its cost (bestCost)
31:    if bestCost = 0 then
32:       break {Stop if a perfect placement is found.}
33:    end if
34:    if genBest(G) = genBest(G − stallGens) then
35:       break {Stop if no improvement in stallGens.}
36:    end if
37: end for
38: return bestNest, bestCost, # of severs used and power consumption.
```

Fig. 3 The newCSA algorithm for VM placement

nests form the top nests. Next, each bottom nest is modified using the Modify_1 perturbation function (Lines 12–15). Note that in Line 13, Modify_1 uses (7) to compute the change in cost of the new solution. This change in cost is used to update the cost of the modified nest in Line 14. For each top nest (Lines 16–28), either Modify_2 or Modify_3 is chosen as perturbation function with equal probability. The *rand* function in Line 17 generates a uniformly distributed random number in the range [0, *x*], where *x* is its argument. The nest resulting from the perturbation is compared with a

randomly chosen nest from the entire population. If the former nest has a lower cost than the latter nest, it replaces it. Lines 29 and 30 update the best cost values found in the current generation and in all generations, respectively. The termination conditions are tested in Lines 31–36. Finally, the best nest, its cost, number of servers utilized, and power consumed are returned in Line 38.

5 Results and discussions

This section discusses the results of experimental tests carried out to verify our proposed algorithm's performance concerning the number of servers required for virtual machine placement, power consumption, and execution time. Power consumption is computed using (6). The values of $P_{\max}$ and P_{idle} are set to 215 watts and 162 watts, respectively [25]. All the algorithms were implemented in MATLAB R2018b and executed on a computer with 131 GB RAM and Intel(R) Xeon(R) CPU E5-2697 v3 2.60GHz (56 CPUs) running 64-bit Windows Server 2012 R2 Data center.

The newly developed CSA (newCSA) is compared with the First Fit Decreasing (FFD) heuristic [27], Best Fit Decreasing (BFD) [28], Reordering Grouping Genetic Algorithm (RGGA) [29] and multiobjective Cuckoo Search Algorithm (multiCSA) [25]. The FFD and BFD heuristics are good representatives of deterministic algorithms for solving VMP. FFD sorts VMs in decreasing order of CPU requirement and tries to place a VM in the first server that can accommodate it. If no existing server can accommodate the current VM, it is placed in a new server. BFD is similar to FFD except that a VM is placed on the server that would produce the smallest leftover resources, rather than on the first server that can accommodate the VM. The RGGA's mutation operation involves a move, remove or swap operator. On the other hand, during the crossover, servers from both parents are combined and sorted based on resource utilization. The more full servers are retained, while those containing VMs belonging to an already selected server are ignored. Finally, VMs that have not been assigned to any server are sorted and placed according to the FFD strategy. multiCSA is a recent multiobjective CSA that tries to simultaneously minimize resource wastage and power consumption. The parameters for RGGA and multiCSA are the same as those used in their original literature. For multiCSA, the population size, fraction of nests to abandon, and the maximum number of iterations were set to 50, 0.75, and 350. In RGGA, the population size, mutation rate, crossover rate, and the maximum number of iterations were set to 75, 0.1, 0.8, and 100, respectively. Table 2 summarizes the algorithm settings.

Two experimental datasets are used to evaluate the proposed CSA. The first dataset labeled A was proposed in [29] based on server consolidation studies by VMware. The dataset consists of nine VM placement instances. The VMP problems in this dataset are designed to be very hard because, in optimal placement of VMs, there are no remnant resources on any server. The CPU and memory capacity of each server was 500. The number of VMs in the nine instances ranges from about 1000 to 6000, so the dataset is a good test of an algorithm's scalability.

The second dataset labeled B is generated by adopting the method used in [27]. The number of VM requests in each instance was fixed at $N = 200$. Each server has

Table 2 Initial parameter settings of algorithms

Algorithm	Parameter	Value
RGGA [29]	Crossover probability	0.8
	Mutation rate	0.1
	Population size	75
	Number of generations	100
MultiCSA [25]	Fraction of bottom nests	0.75
	Population size	50
	Number of generations	350
FFD [27]	Dimension	CPU
BFD [28]	Dimension	CPU
newCSA	Fraction of bottom nests	0.3
	Population size	50
	# of servers to pick in Modify_1 (a)	12
	# of servers to pick in Modify_2 and Modify_3 (b)	12
	Stopping criterion	See Sect. 4.5

a CPU and memory capacity of 100. However, VMs could use only 90 units of CPU and 90 units of memory on each server [25]. A probability P was used to determine whether both CPU and memory requirements for a VM would be higher than or equal to some reference values or if both requirements would be lower than the reference values. The pseudocode of Fig. 4 was used to generate the CPU and memory requirement of each VM. rand(x) is a function that returns a random, uniformly distributed number in the range [0, x). Ref$C$ and Ref$M$ denote the reference values or CPU and memory respectively. P is the probability used to control the correlation between the CPU and memory requirements for each VM.

Two pairs of reference values were utilized: Ref$C$ = Ref$M$ = 25, as well as Ref$C$ = Ref$M$ = 45. Furthermore, five different values of P (0.0, 0.25, 0.5, 0.75 and 1) were used, representing five different correlation levels (strong negative, weak negative, zero, weak positive and strong positive) between memory and CPU requirement. Thus, there were ten combinations of reference values and correlation levels. 100 instances of VM requests were generated for each reference value and correlation level, making a total of 1000 instances, each having 200 VMs. For the case when Ref$C$ = Ref$M$ = 25, the average correlation coefficients between CPU

```
for j=1 to N do
   mv_j ← rand(RefM)
   cv_j ← 2rand(RefC)
   r ← rand(1.0)
   if (r < P and cv_j ≥ RefC) or (r ≥ P and cv_j < RefC) then
      mv_j ← mv_j + RefM
   end if
end for
```

Fig. 4 Pseudocode for generating Dataset B [27]

and memory requirements for the 100 instances were − 0.7525, − 0.3720, 0.0012, 0.3723 and 0.7521. Similarly, when Ref$C$ = Ref$M$ = 45, the corresponding correlation values were − 0.7496, − 0.3757, − 0.0058, 0.3787 and 0.7553.

5.1 Results for dataset A

Except for FFD and BFD which are deterministic algorithms, each of *newCSA*, RGGA, *multiCSA* was run 30 times using the instances of Dataset A. The results of the experiments on Dataset A are presented in Table 3. Column N of the table refers to the number of VMs in an instance, while column OPT M stands for the number of servers required for optimal placement of VMs. FFD and BFD's execution times are a few microseconds in all instances, so they are not shown. It can be observed from Table 3 that *newCSA* almost always finds the optimal placement of VMs. In fact, except for two runs on instance A1, *newCSA* consistently finds an optimal placement all the time. In contrast, FFD always produces placements that require the highest number of servers. BFD performs better than FFD since the latter requires at most two additional servers than the former. *multiCSA* always performs better than RGGA, FFD, and BFD, but worse than *newCSA*. The difference in the performance of the different algorithms increases significantly as the number of VMs increases. For example, when there are about 6,000 VM requests (in A9), *multiCSA*, RGGA and CSA require about 6, 11, and 111 additional servers on the average compared to *newCSA*.

Another advantage of *newCSA* is its execution time. As shown in Table 3, new CSA runs several orders of magnitude faster than *multiCSA* and RGGA, and it can find optimal placements in less than one second, even for A9, which involves about 6,000 VMS. Only FFD and BFD beat the execution speed of *newCSA*. One reason why newCSA is able to obtain optimal solutions in a short time is the manner its perturbation functions work. As discussed in Sect. 4.4, once a server is found to have no remnant resources (i.e., the server's cost is zero), no VMs are ever transferred to/from the server again. This allows the perturbation functions to focus on other servers which still have wasted resources. Secondly, as stated in Sect. 4.2, when a perturbation function is applied to one solution to obtain another solution, it is not required to recompute the cost of the new solution afresh. Instead, the cost of the new solution is obtained from the old cost and the change in costs of only the affected servers. This technique of not computing the cost of new solutions afresh leads to tremendous speedups when there are tens or hundreds of servers.

Moreover, Table 3 shows that even though *newCSA* always results in slightly lower average power consumption than both *multiCSA* and RGGA, it produces placements that lead to great savings in power compared to FFD and BFD. Moreover, as the number of VMs increases, the advantage of *newCSA* over the other algorithms increases. The savings in power range from 151 to 896 W compared to *multiCSA*, 151–1,858 W compared to RGGA, 4,039–17,658 W compared to BFD and 4,039–17,982 W compared to FFD. The reduced power achieved by *newCSA* is a result of its ability to find placements requiring the minimum number of active servers, allowing the idle servers to be switched off.

Table 3 Comparison of experimental results on Dataset A

Case	*N*	OPTM	Alg.	#Servers	#Power	Time(s)
A1	1363	112	newCSA	**112.07**	**24,091**	**0.29**
			multiCSA	113.00	24,242	25.07
			RGGA	113.00	24,242	62.51
			FFD	137.00	28,130	–
			BFD	137.00	28,130	–
A2	2314	191	newCSA	**191.00**	**41,065**	**0.15**
			multiCSA	192.07	41,238	76.15
			RGGA	193.57	41,481	190.28
			FFD	235.00	48,193	–
			BFD	234.00	48,031	–
A3	2639	216	newCSA	**216.00**	**46,440**	**0.15**
			multiCSA	217.03	46,607	112.13
			RGGA	219.10	46,942	246.35
			FFD	265.00	54,378	–
			BFD	264.00	54,216	–
A4	2972	241	newCSA	**241.00**	**51,815**	**0.19**
			multiCSA	242.33	52,031	160.36
			RGGA	244.60	52,398	310.90
			FFD	296.00	60,725	–
			BFD	295.00	60,563	–
A5	3289	267	newCSA	**267.00**	**57,405**	**0.25**
			multiCSA	268.97	57,723	205.34
			RGGA	271.60	58,150	392.38
			FFD	328.00	67,287	–
			BFD	326.00	66,963	–
A6	3958	320	newCSA	**320.00**	**68,800**	**0.30**
			multiCSA	322.33	69,178	317.07
			RGGA	326.10	69,788	470.57
			FFD	395.00	80,950	–
			BFD	393.00	80,626	–
A7	4604	371	newCSA	**371.00**	**79,765**	**0.36**
			multiCSA	373.97	80,246	1255.46
			RGGA	378.93	81,050	365.99
			FFD	457.00	93,697	–
			BFD	455.00	93,373	–
A8	5268	425	newCSA	**425.00**	**91,375**	**0.52**
			multiCSA	429.33	92,077	1185.28
			RGGA	435.07	93,006	1584.85
			FFD	524.00	107,413	–
			BFD	522.00	107,089	–
A9	5949	481	newCSA	**481.00**	**103,415**	**0.73**
			multiCSA	486.53	104,311	1223.87
			RGGA	492.47	105,273	4215.27
			FFD	592.00	121,397	–
			BFD	590.00	121,073	–

Table 3 (continued)

Bold values indicates the algorithm with the best performance (i.e., the algorithm with the lowest number of servers, power consumption or execution time in each experiment)

Figure 5 shows how the cost value of the best solution found by newCSA changes with time for three instances of Dataset A. Recall that the cost of an optimal solution (using (7) and (8)) is zero. This graph shows the short convergence time of the proposed newCSA algorithm.

5.2 Results for dataset B

The results of the experiments on Dataset B are shown in Table 4. Again, BFD and FFD's execution times are not shown because they take only a few microseconds. Since the instances of VMPP are randomly generated using the algorithm of Fig. 4, the minimum number of servers required in an optimal placement is not known. Therefore, the average theoretical lower bound (LB) on the number of servers required is shown in the table. *LB* for any instance of VMP problem is computed using [25]:

$$\text{LB} = \max\left\{\left\lceil \sum_{j=1}^{N} \text{cv}_j/\text{CS}_i \right\rceil, \left\lceil \sum_{j=1}^{N} \text{mv}_j/\text{MS}_i \right\rceil\right\}$$

From Table 4, we observe the following:

– newCSA always outperforms RGGA, multiCSA, BFD, and FFD in terms of the average number of servers required, except in the two cases where there is a

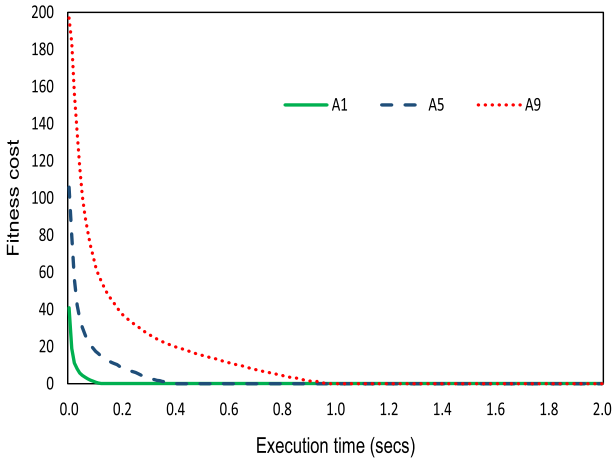


Fig. 5 Convergence characteristics of newCSA for three instances of Dataset A

Table 4 Comparison of experimental results on Dataset B

Ref.	Corr.	Alg.	LB	#Servers	#Power	Time(s)
25, 25	Strong -ve	newCSA	57.18	60.17	12,383	**0.1920**
		multiCSA		**59.33**	**12,246**	1.7152
		RGGA		60.14	12,378	0.96
		FFD		80.34	15,650	–
		BFD		80.04	15,601	–
	Weak -ve	newCSA	56.99	**58.82**	**12,180**	**0.2681**
		multiCSA		58.89	12,191	1.0147
		RGGA		59.43	12,279	0.95
		FFD		73.48	14,791	–
		BFD		73.48	14,555	–
	Zero	newCSA	56.80	**58.31**	**12,086**	**0.2896**
		multiCSA		58.51	12,119	0.9957
		RGGA		58.97	12,193	0.93
		FFD		70.04	13,987	–
		BFD		68.46	13731	–
	Weak +ve	newCSA	56.57	**57.89**	**12,033**	**0.2919**
		multiCSA		58.27	12,094	1.0090
		RGGA		58.67	12,159	0.88
		FFD		65.75	13,306	–
		BFD		64.69	13,134	–
	Strong +ve	newCSA	55.96	**57.10**	**11,892**	**0.2801**
		multiCSA		57.53	11,962	0.9938
		RGGA		57.73	11,994	0.86
		FFD		61.07	12,535	–
		BFD		60.29	12,409	–

Table 4 (continued)

Ref.	Corr.	Alg.	LB	#Servers	#Power	Time(s)
45, 45	Strong -ve	newCSA	102.65	121.38	24,420	**0.0761**
		multiCSA		**121.32**	**24,410**	1.8731
		RGGA		121.79	24,486	0.67
		FFD		132.84	26,276	–
		BFD		122.43	24,590	–
	Weak -ve	newCSA	102.95	**118.66**	**24,009**	**0.1069**
		multiCSA		118.71	24,017	1.8099
		RGGA		119.45	24,137	0.51
		FFD		132.4	26,235	–
		BFD		120.68	24,337	–
	Zero	newCSA	102.42	**116.16**	**23,590**	**0.1497**
		multiCSA		116.52	23,648	1.8237
		RGGA		117.31	23,776	0.51
		FFD		130.31	25,882	–
		BFD		119.14	24,072	–
	Weak +ve	newCSA	101.79	**112.22**	**22,948**	**0.1734**
		multiCSA		113.29	23,122	1.7663
		RGGA		113.97	23,232	0.50
		FFD		124.27	24,900	–
		BFD		115.96	23,554	–
	Strong +ve	newCSA	101.25	**108.70**	**22,380**	**0.1792**
		multiCSA		110.02	22,594	1.7310
		RGGA		110.59	22,686	0.51
		FFD		115.15	23,424	–
		BFD		110.50	22,673	–

Bold values indicates the algorithm with the best performance (i.e., the algorithm with the lowest number of servers, power consumption or execution time in each experiment)

strong negative correlation between CPU and memory requirement of VMs. In both these cases, the best results are obtained by multiCSA.

– Other than FFD and BFD, the average running time for newCSA is always lower than that of the other algorithms. Thus, the good performance of newCSA is obtained in comparatively little time.

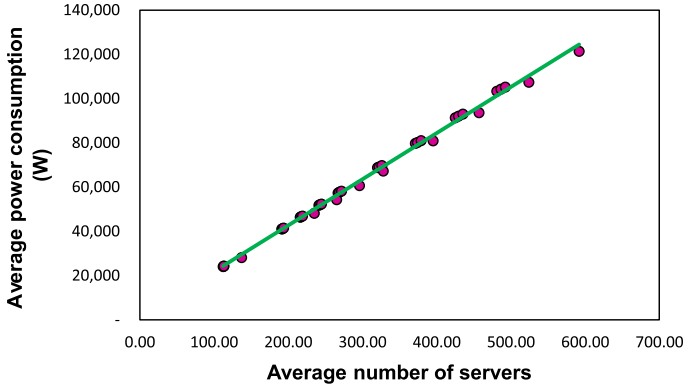
- BFD always performs better than FFD in terms of the number of servers required and power consumption, even though the former requires slightly more computation time than the latter.
- The average power consumption of placements found by multiCSA is lowest in the two cases when a strong negative correlation exists between CPU and memory requirement. In all the remaining eight cases, newCSA consistently resulted in lower power consumption than all the remaining algorithms.
- In the case when Ref$C$ = Ref$M$ =25 and when Ref$C$ = Ref$M$ = 45, the number of servers needed for placement decreases as the degree of correlation increases from strong negative to strong positive. This is because a negative correlation between CPU and memory requirement leads to lots of wastage of server resources, resulting in additional servers. Furthermore, placements for instances where Ref$C$ = Ref$M$ = 25 generally require fewer servers than those where Ref$C$ = RefM = 45 because in the former cases, VM requests are in the range [0, 50) while in the latter case, they fall within the range [0, 90).
- In all the ten cases, the algorithm producing the least number of servers also had the lowest power consumption. To investigate the relationship between the number of servers used for placement and power consumption, we produced plots of the average number of servers vs. corresponding power consumption shown in Tables 3 and 4. The plots are shown in Fig. 6. From the high degree of correlation between the number of servers and power consumption, we can conclude that minimizing the number of servers in placement is a good way of reducing power consumption.

5.2.1 Convergence plot

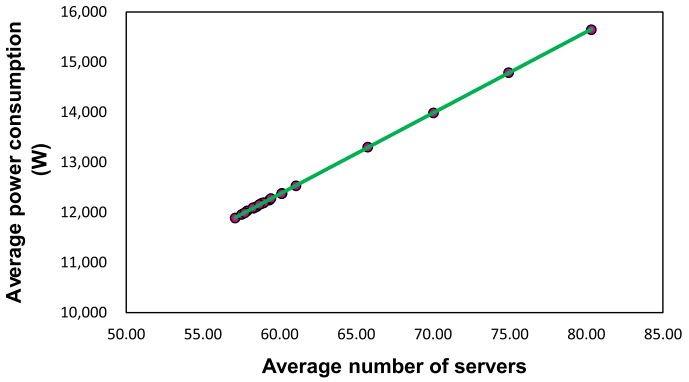
Figure 7 shows the trajectory of the decrease in the number of servers with time for some selected data. Without loss of generality, for Dataset A, A9 was chosen, and for Dataset B, we selected the Ref$C$= Ref$M$ = 45 (strong negative correlation). The plot is the average of ten runs of each algorithm. From the plots, it can be observed that the newCSA has better convergence compared to the other methods.

5.2.2 Scalability Test

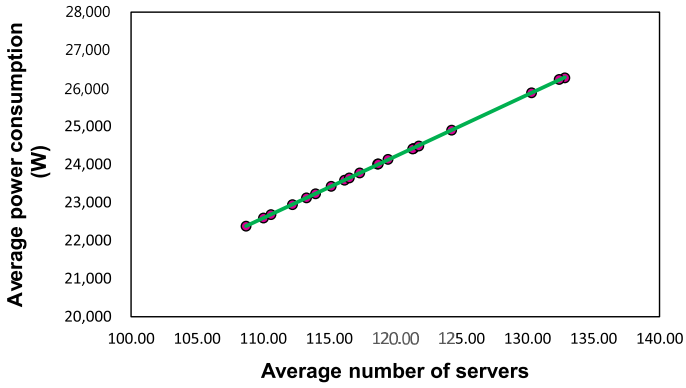
To test the scalability of newCSA, the average execution time of the algorithm was measured for VMPPs involving a larger number of VMs. The number of VMs was varied from 200 to 2000 in steps of 200 for the correlation level $P = 0.0$, representing a strong negative correlation. This correlation level was chosen because it is the most difficult placement due to the strong negative correlation between the CPU and memory resources. Figure 8 shows that newCSA requires at most 13 s, even in the case when the number of VMs is 2,000. This suggests that newCSA is suitable for large-scale VM placements.



(a) Dataset A Corr. Coeff. = 0.998425

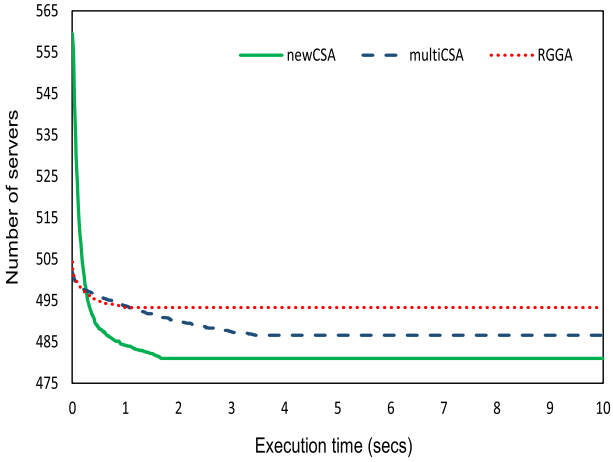


(b) Dataset B (RefC = RefM = 25) Corr. Coeff. = 0.999974

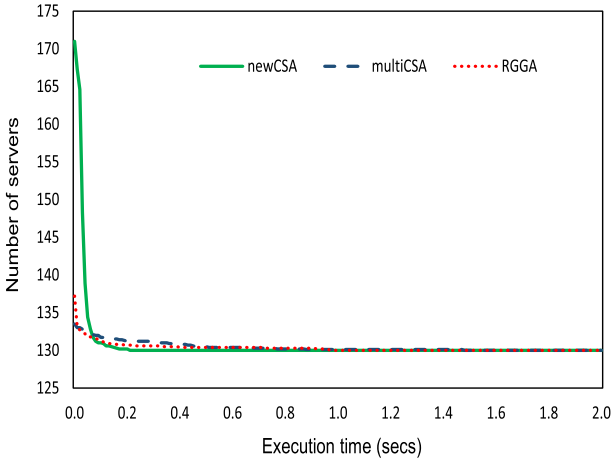


(c) Dataset B (RefC = RefM = 45) Corr. Coeff. = 0.999963.

Fig. 6 Correlation between number of servers and power consumption on different datasets



(a) Dataset A (A9)



(b) Datset B (Ref = 45, strong negative corr.)

Fig. 7 A plot of showing how the number of servers decreases with time for selected dataset instances. It compares the newCSA with other techniques

5.2.3 Statistical significance

In order to statistically verify the superior performance of newCSA over the other algorithms, the Wilcoxon's rank sum test was carried out at 5% significance level for newCSA vs each of the other algorithms. The results of the rank sum tests are shown in Tables 5 and 6 for Dataset A and Dataset B, respectively. In Table, "1", "0" and "− 1" signify that newCSA is better, similar or worse than another algorithm at 95% confidence level, respectively.

As shown in Table 5, for Dataset A, newCSA is always significantly better than the other algorithms in terms of number of servers required for VM

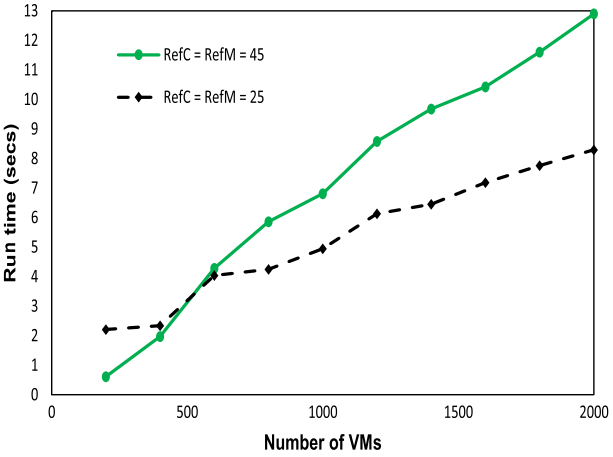


Fig. 8 A figure showing the scalability of the newCSA

Table 5 Results of Wilcoxon's rank sum tests at 5% significance comparing newCSA with other methods on Dataset A

Instance	Number of servers				Power			
	FFD	BFD	RGGA	multiCSA	FFD	BFD	RGGA	multiCSA
A1	1	1	1	1	1	1	1	1
A2	1	1	1	1	1	1	1	1
A3	1	1	1	1	1	1	1	1
A4	1	1	1	1	1	1	1	1
A5	1	1	1	1	1	1	1	1
A6	1	1	1	1	1	1	1	1
A7	1	1	1	1	1	1	1	1
A8	1	1	1	1	1	1	1	1
A9	1	1	1	1	1	1	1	1

1 = newCSA is better, 0 = no significant difference, − 1 = newCSA is worse

placement and power consumption. In the case of Dataset B, the results in Table 6 show that newCSA is significantly better than the other algorithms in terms of power consumption. However, in terms of number of servers, newCSA significantly outperforms the other algorithms except multiCSA. It is worth noting that the problem instances in Dataset A ($N = 1363 - 5949$ VMs) are much larger than those in Dataset B ($N = 200$); thus, the advantage of newCSA over the other algorithms is more pronounced in the former dataset. Moreover, server cost is a one-off cost, while power costs are recurrent costs for data centers.

Table 6 Results of Wilcoxon's rank sum tests at 5% significance comparing newCSA with other methods on Dataset B

(RefC, RefM)	Correlation	Number of servers				Power			
		FFD	BFD	RGGA	multiCSA	FFD	BFD	RGGA	multiCSA
(25, 25)	Strong neg.	1	1	0	− 1	1	1	− 1	− 1
	Weak neg.	1	1	1	0	1	1	1	1
	Zero	1	1	1	0	1	1	1	1
	Strong pos.	1	1	1	0	1	1	1	1
	Weak pos.	1	1	1	0	1	1	1	1
(45, 45)	Strong neg.	1	1	0	0	1	1	1	− 1
	Weak neg.	1	1	0	0	1	1	1	1
	Zero	1	1	0	0	1	1	1	1
	Strong pos.	1	1	1	0	1	1	1	1
	Weak pos.	1	1	1	1	1	1	1	1

1 = newCSA is better, 0 = no significant difference, − 1 = newCSA is worse

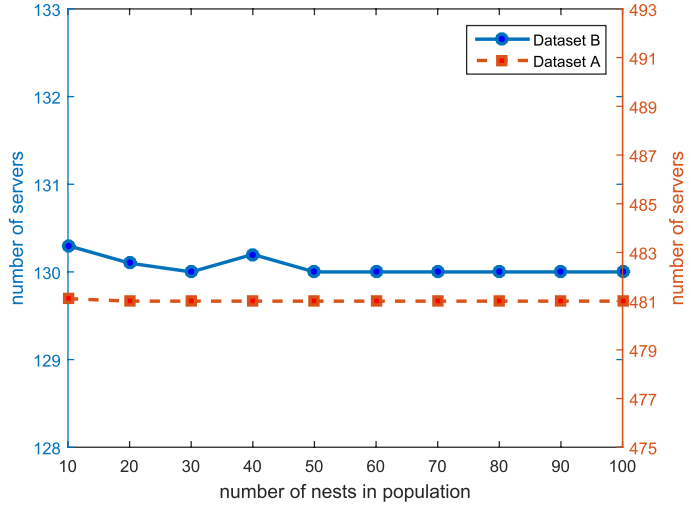
5.2.4 Analysis of newCSA's parameters

This section investigates the main parameters of newCSA which are the population size, the fraction of population in the bottom nests, no. of servers to pick in Modify_1 (a), and no. of servers to pick in Modify_2 and Modify_3 (b). In order to observe the effect of the parameters on the quality of solutions obtained, A9 was chosen from Dataset A, while from Dataset B we selected the Ref$C$ = Ref$M$ = 45 (strong negative correlation). While investigating a particular parameter, the other parameters take the values given in Table 2.

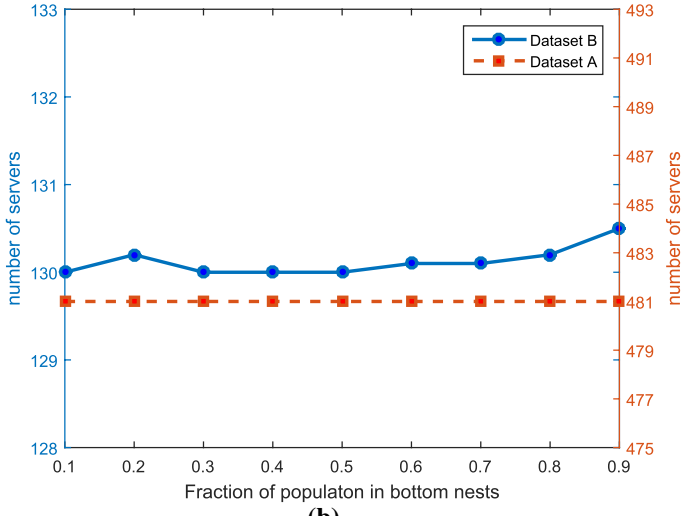
The first parameter investigated is population size, which is varied from 10 to 100 in steps of 10. The average number of servers obtained for different population sizes is given in Fig. 9a. It can be seen that variation of population size minimally affects the performance of newCSA. For Dataset A, population size has little or no impact on number of servers required. For Dataset B, population sizes of at least 50 produce the best results. However, the larger the population size, the more the computation required in each generation. Therefore, population size of 50 was used in our experiments.

The next parameter examined is fraction of population forming bottom nests. The average number of servers obtained by varying the proportion of bottom nests from 0.1 to 0.9 is shown in Fig. 9b. From the figure, proportion of bottom nests does not have a major impact on our algorithm's results. For Dataset A, fraction of bottom nests hardly affects the algorithm's performance. For Dataset B, bottom nests of between 0.3 and 0.5 produce optimal results. However, outside this range, the number of servers required marginally increased from 130 to at most 130.5. Thus, the fraction of bottom nest was fixed at 0.3 for our experiments.

Next, the impact of no. of servers (a) selected during bottom nest perturbation i.e., during Modify_1 is investigated. As shown in Fig. 9c, for Dataset A, values of parameter a above 4 produced optimal results. Other values of a increased the



(a)



(b)

Fig. 9 Influence of parameters on performance of newCSA

average number of servers from 481 to at most 481.9. For Dataset B, values of a between 10 and 12 slightly increased the average number the servers from 130 to 130.1. Therefore, parameter a has little impact on the performance of newCSA. In all experiments, the value of a was set to 12.

The final parameter that was examined is the of number of servers (b) selected during top nest perturbations i.e., during Modify_2 and Modify_3. As shown in

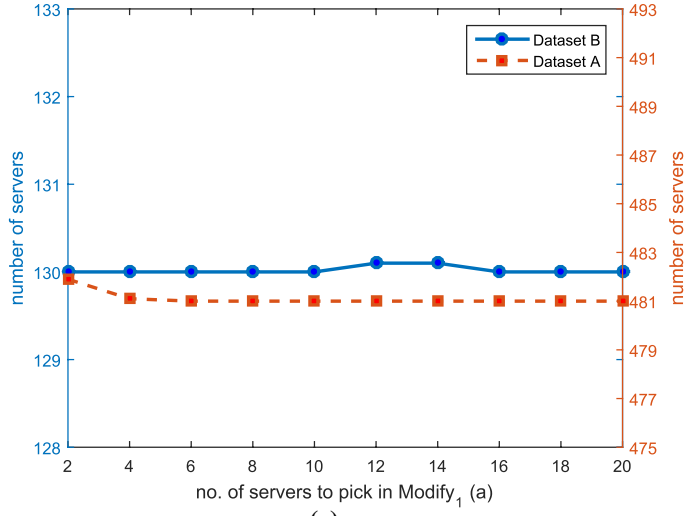


Fig. 9 (continued)

Fig. 9d, b had the greatest impact on our algorithm. For Dataset A, low values of parameter b (i.e., $b < 8$) significantly increased the number of servers required from 481 to at most 493. However, higher values of b produced optimal results. Similarly, low values of b worsened the performance of newCSA on Dataset B, although in this case the average number the servers marginally increased from

130 to 130.6 in the worst case. Accordingly, the value of b was set to 12 for all experiments.

6 Conclusions

We have implemented a new cuckoo search (CS) optimization technique to solve the virtual machine (VM) placement problem of data centers. The main target is to place VMs in physical machines such that a minimum number of machines are used. The result we have obtained from testing the technique on two standard benchmarks showed the technique's robustness. The new CS method was better than reordered grouping genetic algorithm (RGGA), first-fit decreasing (FFD), best-fit decreasing (BFD), and a prior CS method termed multiCSA. Great future directions may include hybridizing the new CS with other metaheuristics to improve performance. Moreover, since we deal with only two dimensions of CPU and memory, it would be great to employ the new CS technique in VM placement with three or more resources (dimensions).

Acknowledgements We acknowledge all support from King Fahd University of Petroleum & Minerals (KFUPM), Saudi Arabia, University of Hafr al Batin, Saudi Arabia, and Universiti Teknologi PETRONAS (UTP), Malaysia. The authors are also grateful to Prof. Kevin Seppi of Brigham Young University, USA, for providing the VMPP instances in Dataset A.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

1. Varghese B, Buyya R (2018) Next generation cloud computing: new trends and research directions. Future Gener Comput Syst 79:849–861
2. Etro F (2015) The economics of cloud computing. In: IRMA (ed) Cloud technology: concepts, methodologies, tools, and applications, IGI global, Hershey. https://doi.org/10.4018/978-1-4666-6539-2.ch101
3. Rittinghouse John W, Ransome James F (2016) Cloud computing: implementation, management, and security. CRC Press, Florida
4. Martello S, Pisinger D, Vigo D (2000) The three-dimensional bin packing problem. Oper Res 48(2):256–267
5. Falkenauer E (1996) A hybrid grouping genetic algorithm for bin packing. J Heuristics 2(1):5–30
6. Yang X-S, Deb S (2009) Cuckoo search via Lévy flights. In: 2009 world congress on nature & biologically inspired computing (NaBIC), Coimbatore. https://doi.org/10.1109/NABIC.2009.5393690
7. Walton S, Hassan O, Morgan K, Brown MR (2011) Modified cuckoo search: a new gradient free optimisation algorithm. Chaos, Solitons Fractals 44(9):710–718
8. Cid-Garcia Nestor M, Rios-Solis Yasmin A (2020) Positions and covering: a two-stage methodology to obtain optimal solutions for the 2d-bin packing problem. Plos One 15(4):e0229358
9. Ross N, Keedwell E, Savic D (2020) Human-derived heuristic enhancement of an evolutionary algorithm for the 2d bin-packing problem. In: International Conference on Parallel Problem Solving from Nature, Springer. 413–427

10. Dell'Amico M, Furini F, Iori M (2020) A branch-and-price algorithm for the temporal bin packing problem. Comput Oper Res 114:104825104825104825
11. Yao Y, Lai C, Cui Y (2017) A constructive heuristic for the two-dimensional bin packing based on value correction. Int J Comput Appl Technol 55(1):12–21
12. Bennell JA, Cabo M, Martinez-Sykora A (2018) A beam search approach to solve the convex irregular bin packing problem with guillotine cuts. Eur J Oper Res 270(1):89–102
13. Sato AK, Martins TC, Tsuzuki MSG (2018) Obstruction map local search solution for 2d irregular bin packing problem with cache acceleration. In: *2018 13th IEEE International Conference on Industry Applications (INDUSCON)*, IEEE. 837–843
14. Mustafa S, Bilal K, Madani SA, Tziritas N, Khan SU, Yang LT (2015) Performance evaluation of energy-aware best fit decreasing algorithms for cloud environments. In: *2015 IEEE International Conference on Data Science and Data Intensive Systems*, IEEE. 464–469
15. Adamuthe Amol C, Pandharpatte Rupali M, Thampi Gopakumaran T (2013) Multiobjective virtual machine placement in cloud environment. In: *2013 International Conference on Cloud & Ubiquitous Computing & Emerging Technologies*, IEEE. 8–13
16. Xu J, Fortes Jose AB (2010) Multi-objective virtual machine placement in virtualized data center environments. In: *2010 IEEE/ACM Int'l Conference on Green Computing and Communications & Int'l Conference on Cyber, Physical and Social Computing*, IEEE. 179–188
17. Perumal B, Murugaiyan A (2016) A firefly colony and its fuzzy approach for server consolidation and virtual machine placement in cloud datacenters. Adv Fuzzy Syst 2016:5
18. Wu Y, Tang M, Fraser W (2012) A simulated annealing algorithm for energy efficient virtual machine placement. In: *2012 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, IEEE. 1245–1250
19. Gao Y, Guan H, Qi, Hou Y, Liu L (2013) A multi-objective ant colony system algorithm for virtual machine placement in cloud computing. J Comput Syst Sci 79(8):1230–1242
20. Wang S, Liu Z, Zheng Z, Sun Q, Yang F (2013) Particle swarm optimization for energy-aware virtual machine placement optimization in virtualized data centers. In: 2013 International Conference on Parallel and Distributed Systems, IEEE. 102–109
21. Alboaneen DA, Tianfield H, Zhang Y (2016) Glowworm swarm optimisation algorithm for virtual machine placement in cloud computing. In: 2016 Intl IEEE Conferences on Ubiquitous Intelligence & Computing, Advanced and Trusted Computing, Scalable Computing and Communications, Cloud and Big Data Computing, Internet of People, and Smart World Congress (UIC/ATC/ScalCom/CBDCom/IoP/SmartWorld), IEEE. 808–814
22. Patel P, Ranabahu AH, Sheth AP (2009) Service level agreement in cloud computing. https://corescholar.libraries.wright.edu/knoesis/78. Accessed 18 Jan 2021
23. Abdel-Basset M, Abdle-Fatah L, Sangaiah AK (2018) An improved Lévy based whale optimization algorithm for bandwidth-efficient virtual machine placement in cloud computing environment. Clust Comput 22(4):8319–8334
24. Li Z, Li Y, Yuan T, Chen S, Jiang S (2019) Chemical reaction optimization for virtual machine placement in cloud computing. Appl Intell 49(1):220–232
25. Sait SM, Bala A, El-Maleh AH (2016) Cuckoo search based resource optimization of datacenters. Appl Intell 44(3):489–506
26. Liu X-F, Zhan ZH, Deng JD, Li Y, Gu T, Zhang J (2016) An energy efficient ant colony system for virtual machine placement in cloud computing. IEEE Trans Evolt Comput 22(1):113–128
27. Ajiro Y, Tanaka A (2007) Improving packing algorithms for server consolidation. Int CMG Conf 253:399–406
28. Sgall J (2012) A new analysis of best fit bin packing. In: International Conference on Fun with Algorithms, Springer. 315–321
29. Wilcox D, McNabb A, Seppi K (2011) Solving virtual machine packing with a reordering grouping genetic algorithm. In: *2011 IEEE Congress of Evolutionary Computation (CEC)*, IEEE 362–369

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Hamza Onoruoiza Salami[1] · Abubakar Bala[2,3] · Sadiq M. Sait[4] · Idris Ismail[2]

Abubakar Bala
abala.ele@buk.edu.my

Sadiq M. Sait
sadiq@kfupm.edu.sa

Idris Ismail
idrisim@utp.edu.my

[1] College of Computer Science and Engineering, University of Hafr al Batin, Hafr al Batin 31991, Saudi Arabia

[2] Department of Electrical and Electronics Engineering, Universiti Teknologi PETRONAS, Malaysia, 32610, Bandar Seri Iskandar, Perak, Malaysia

[3] Electrical Engineering Department, Bayero University Kano, PMB 3011, Gwarzo Road, Kano, Nigeria

[4] Computer Engineering Department and Center for Communications and IT Research, Research Institute, King Fahd University of Petroleum & Minerals, Dhahran 31261, Saudi Arabia